Practical - 3

Mayur Zope BE Comp A 71

To solve the fractional knapsack problem using a greedy algorithm.

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Item {
   int weight;
   int value;
};

bool compare(Item a, Item b) {
   double r1 = (double)a.value / a.weight;
   double r2 = (double)b.value / b.weight;
   return r1 > r2;
}

int main() {
   Item items[] = {
      {10, 60},
      {20, 100},
      {30, 120}
   };

   int n = 3;
   int capacity = 50;

   sort(items, items + n, compare);

   double totalValue = 0;
```

```
    for (int i = 0; i < n; i++) {
        if (capacity == 0)
            break;

        if (items[i].weight <= capacity) {
            capacity -= items[i].weight;
            totalValue += items[i].value;
        } else {
            totalValue += (double)items[i].value
                    * capacity / items[i].weight;
            capacity = 0;
        }
    }

    cout << "Maximum Value: " << totalValue << endl;

    return 0;
}
```

//OUTPUT

